

交通信号灯监控设备通信协议

V 0.1.3

2026 年 4 月 2 日

中国科大-先进仪器实验室

<http://ail.ustc.edu.cn>



版本历史

版本	发布日期	描述	备注
V0.1.1	2026/03/11	首次发布	
V0.1.2	2026/03/25	增加 ledState	参见“EVENT_PAK 格式定义”部分修改
V0.1.3	2026/04/02	增加参数配置内容	参见 4 参数配置

前 言

本文档主要说明上位机和监控设备之间的通信协议，协议基于 MQTT 3.1.1 版本协议。

每一个节点仪器建立两个主题实现和主机之间的双向通信，“trafficLight/networkCode/stationCode/ledHwId/U”和“trafficLight/networkCode/stationCode/ ledHwId /D”分别对应节点向主机发送的数据和主机向节点发送的数据。U 表示 up-stream 上行数据，即节点向主机方向传送的数据；D 表示 down-stream 下行数据，即主机向节点发出的数据或命令。其中 networkCode、stationCode 和 ledHwId 都是 16 进制**大写**十六进制 ASCII 码，例如 trafficLight/0/0/11130000/U，trafficLight/0/0/11130000/D。其中 networkCode 表示地区编号（0~254），stationCode 表示工作区编号(0-65534)，目前这两项默认均为 0，不建议修改。ledHwId 是设备的固有编号，不可以自定义，由出厂时设定。

目录

前 言	3
1. 工作方式	5
2. 信号灯监控设备工作流程	5
3. 通信协议	5
3.1 基本命令帧	6
3.1.1 COMMAND 定义	7
3.1.2 EVENT_PAK 定义	7
3.1.3 FIRMWARE_PAK 格式定义	9
3.1.4 FIRMWARE_INFO_DAT 格式定义	9
3.1.5 FIRMWARE_PAK 格式定义	10
3.2 CMD_CHECKIN 命令	10
3.3 CMD_ALARM 命令	11
3.4 CMD_POWER_ON 命令	11
3.5 CMD_CHECK_FW 命令	11
4. 参数配置	12
4.1 mqtt.ini 结构解释	12
4.2 trafficLight.ini 结构解释	13
4.3 trafficLightConf 使用方法	13
5. 附录	16
1.1 校验算法	16

1. 工作方式

整个系统的工作模式如下图所示。信号灯监控设备在检测到故障或签到周期结束时会主动向 MQTT 服务器发送数据包，上位机通过服务器接收设备节点发送的数据，需要时也可通过 MQTT 服务器发送命令给设备节点。

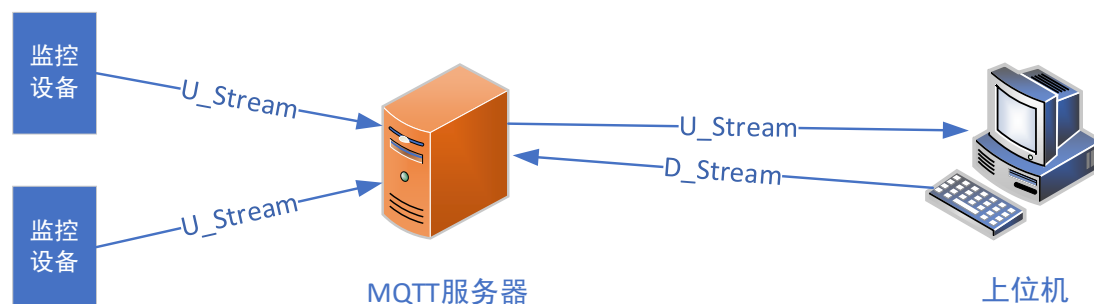


图 1 系统结构拓扑图

2. 信号灯监控设备工作流程

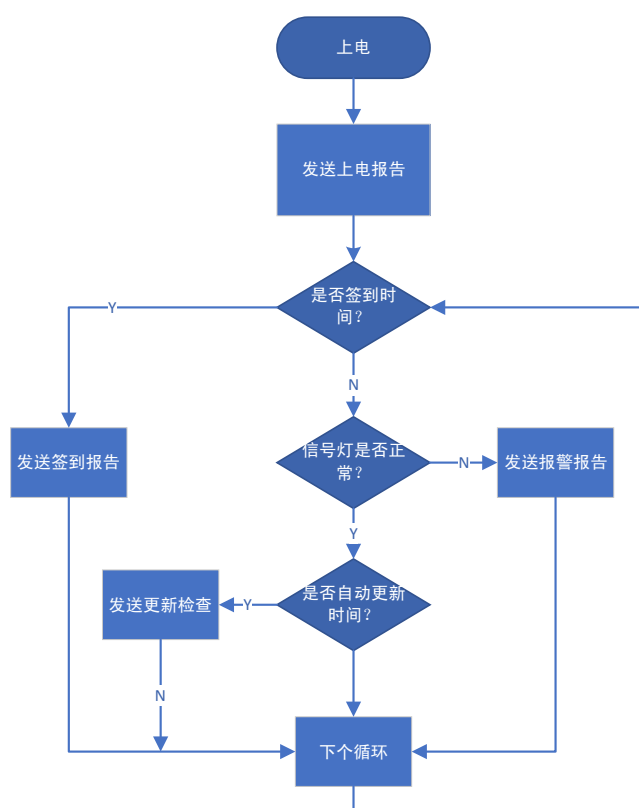


图 1 设备工作流程示意图

3. 通信协议

3.1 基本命令帧

基本通信命令格式定义如下

```
typedef struct {
    uint8_t    token;           //a valid cmd token should be 0x55
    uint8_t    cmd;             //@ref COMMAND
    uint8_t    ver;             //协议版本号, 0x10 = v1.0
    uint8_t    checksum;        //校验和, 数据包所有内容按无符号数 (uint8) 相加必须等于0xff
    uint32_t    swVer;          //软件版本号
    uint16_t    cmdSeq;         //包序号, 每发送一个命令, 序号加一。数据帧和命令帧序号独立计数
    uint16_t    datLen;         //包数据的长度 (cmdFrame.dat部分的长度)
    uint32_t    userVal;        //用户自定义数据, 可用于简短的数据交互
    //以上为数据头, 下面开始为数据内容, 根据实际需要长度可变
    uint8_t    dat[0];         //can be EVENT_PAK, FIRMWARE_PAK, REG_DAT
}CMD_FRAME ;
```

表 3-1 命令帧格式解释

字节与位序号	数据类型	名称	备注
00	uint8_t	token	魔术字, 0x55
01	uint8_t	cmd	命令类型, 参见下面 COMMAND 定义解释
02	uint8_t	ver	协议的版本号, 目前为 0x10
03	uint8_t	checksum	校验和, 整个数据包所有内容按无符号数 (uint8_t) 相加必须等于 0xff
04-07	uint32_t	swVer	设备的软件版本号 bit[31:28] = major version bit[27:24] = minor version bit[23:18] = build year (2000+n) bit[17:14] = build month bit[13:8] = build day bit[7:0] = build number (begin from 1 if a new day begin)
08~09	uint16_t	cmdSeq	包序号, 每发送一个命令, 序号加一
10~11	uint16_t	datLen	包数据的长度 (CMD_FRAME->dat 部分的长度, bytes)
12-15	uint32_t	userVal	用户自定义数据, 可用于简短的数据交互
16-xx	uint8_t	dat	根据 cmd 的不同, 该部分有不同的长度和定义。目前有 EVENT_PAK、FIRMWARE_PAK、FIRMWARE_INFO_DAT、FIRMWARE_PAK 几种不同的数据结构定义, 具体见下面定义

3.1.1 COMMAND 定义

```
#define CMD_CHECKIN      0x00    //checkin
#define CMD_ALARM        0x01    //alarm (some value is beyond the threshold)
#define CMD_POWER_ON     0x03    //power ON
#define CMD_UPDATE_CONFIG 0x20    //set new config to node
#define CMD_CHECK_FW      0x30    //check if new fw ready?
#define CMD_GET_FW        0x31    //require firmware update from server
#define CMD_REBOOT       0x7f    //reboot the node

#define CMD_ACK_FLAG      0x80    //bit7: 1=回应帧, 0=非回应帧
```

表 3-2 COMMAND定义

编码	帧命令	功能说明
0x00	CMD_CHECKIN	签到数据，节点按照设定周期向服务器签到，表示设备工作正常
0x01	CMD_ALARM	报警数据，节点发现信号灯异常
0x03	CMD_POWER_ON	上电或重启后，节点向服务器报告启动完成
0x20	CMD_UPDATE_CONFIG	服务器通过该命令向节点发送配置更新指令
0x30	CMD_CHECK_FW	节点向服务器查询是否有新固件可更新
0x31	CMD_GET_FW	节点向服务器发出请求固件更新数据
0x7f	CMD_REBOOT	服务器通过该命令向节点发送重启指令

注：帧命令定义为 0x00-0x7f。最高位=0 表示发出的请求，最高位=1 表示该帧是回应帧

3.1.2 EVENT_PAK 定义

当 CMD_FRAME->cmd 为 CMD_CHECKIN 或 CMD_ALARM 类型时，命令帧的 dat 部分按 EVENT_PAK 格式解释。

```
typedef struct {
    uint16_t eventRecordNum;
    uint16_t datLen;           //eventRecord部分的长度, bytes
    EVENT_RECORD eventRecord[0]; //@ref EVENT_RECORD
}EVENT_PAK;

typedef struct {
```

```

uint32_t    ledHwId;        //灯组设备的hwId
uint32_t    subHwId;        //同一设备下不同灯组的ID
uint32_t    swVer;          //软件版本号
uint32_t    confVer;        //配置版本号

uint8_t     ledState;        //给出当前灯组的亮灯状态 @ref LED_STATE
int8_t      errCode;         //@ref LED_ERR_CODE
uint16_t    current[3];     //电流大小
}EVENT_RECORD;

```

1. EVENT_PAK 格式定义

具体格式定义如上，解释如下：

名称	解释
eventRecordNum	表示总共包含多少个 event 数据
datLen	表示 EVENT_RECORD eventRecord[0] 总共有多少字节内容
eventRecord	根据 eventRecordNum，该部分可能包含 1 条或者多条 EVENT 数据，每条 EVENT 的格式按上 EVENT_RECORD 解释。

2. EVENT_RECORD 格式定义

具体格式定义如上，解释如下：

名称	解释
ledHwId	表示设备的硬件 ID，每个设备唯一，由出厂时设定不可更改
subHwId	同一设备下不同灯组的 ID，适用于管理多组灯组（目前暂未定义，后续会扩展）
swVer	设备的固件版本号，参见表 3-1 命令帧格式解释
confVer	配置参数的版本号，
	按 16 进制定义 0xYYMMDDnn YYMMDD=日期，nn=当天版本 例如 0x26030801=26 年 3 月 8 日 01 版
reserved	保留
errCode	错误类型，具体定义见表 3-3 errCode 定义
current[3]	红黄绿三个信号灯的电流值，范围：0-2048。相对值有意义，可通过历史数据反映电流变化趋势，缺亮一般会导致该值变大。

表 3-3 errCode 定义

名称	数值	解释
LED_ERR_OK	0	工作正常，无错误
LED_ERR_ROFF	-1	红灯周期全灭（当前处于红灯周期，但检测到所有灯全灭）
LED_ERR_YOFF	-2	黄灯周期全灭（当前处于黄灯周期，但检测到所有灯全灭）
LED_ERR_GOFF	-3	绿灯周期全灭（当前处于绿灯周期，但检测到所有灯全灭）
LED_ERR_RYON	-4	红黄同亮

LED_ERR_RGON	-5	红绿同亮
LED_ERR_YGON	-6	黄绿同亮
LED_ERR_RYGON	-7	红黄绿同亮
LED_ERR_RON_TIMEOUT	-8	红灯亮灯超过设定时间
LED_ERR_YON_TIMEOUT	-9	黄灯亮灯超过设定时间
LED_ERR_GON_TIMEOUT	-10	绿灯亮灯超过设定时间
LED_ERR_RDIM	-11	红灯缺亮（暂未实现）
LED_ERR_YDIM	-12	黄灯缺亮（暂未实现）
LED_ERR_GDIM	-13	绿灯缺亮（暂未实现）
LED_ERR_POWER_LOSS	-14	断电（超过设置时间阈值）

表 3-4 LED_STATE 定义

名称	数值	解释
STATE_R	0	灯组处于红灯状态
STATE_Y	1	灯组处于黄灯状态
STATE_G	2	灯组处于绿灯状态
STATE_NONE	-1	未知状态（由于故障，无法确定灯组处于什么状态）

3.1.3 FIRMWARE_PAK 格式定义

当 CMD_FRAME->cmd 为 CMD_GET_FW|0x80 时, 命令帧的 dat 部分按 FIRMWARE_PAK 格式解释。该数据帧用于服务器向节点设备发送固件升级包。

```
typedef struct {
    uint8_t    target;    //升级包目标站体，可自行编号0,1,2,3...
    uint8_t    datLen;    //dat部分的长度（数据长度 = datLen+1 Bytes。除了最后一个包，其余
                          //的数据包都应该为最大包长255+1）
    uint16_t   offset;    //当前数据对应升级包的偏移量
    uint8_t    dat[0];    //升级包数据
}FIRMWARE_PAK;
```

3.1.4 FIRMWARE_INFO_DAT 格式定义

当 CMD_FRAME->cmd 为 CMD_CHECK_FW|0x80 时，表示服务器回应设备更新查询命令帧数据，此时命令帧的 dat 部分该定义解释。

```
typedef struct {
    uint32_t    swVer;    //固件版本号
    uint16_t    fwLen;    //固件的长度（字节数）
    uint8_t     fwChecksum; //固件的校验和，整个数据包所有内容按无符号数（uint8_t）相加
    uint8_t     reserved;  //保留
}FIRMWARE_INFO_DAT;
```

3.1.5 FIRMWARE_PAK 格式定义

当 CMD_FRAME->cmd 为 CMD_GET_FW|0x80 时，表示服务器回应设备请求的固件更新数据，此时命令帧的 dat 部分该定义解释。

```
typedef struct {
    uint8_t    target;    //升级包目标站体，可自行编号0,1,2,3...（目前未启用）
    uint8_t    datLen;    //dat部分的长度（数据长度 = datLen+1 Bytes。除了最后一个包，其余的数据包都应该为最大包长）

    uint16_t   offset;    //当前数据对应升级包的偏移量
    uint8_t    dat[0];    //升级包数据（1-256bytes）
}FIRMWARE_PAK;
```

3.2 CMD_CHECKIN 命令

该命令由节点设备定时发送给服务器，服务器收到该命令后需将系统时间返回给节点设备用于设备本地时间的同步。时间信息通过 userVal 返回，其定义为 epoch seconds，即 1970 年 1 月 1 日 00:00:00 UTC 以来的秒数+时区修正（UTC+8x3600）。

举例：

- 1) 设备发送数据包 CMD_FRAME+ EVENT_PAK+ EVENT_RECORD 给服务器（对于目前版本设备，只有一条 EVENT_RECORD）。

```
uint8_t    token;
uint8_t    cmd = CMD_CHECKIN
uint8_t    ver;
uint8_t    checksum;
uint32_t   swVer;
uint16_t   cmdSeq;
uint16_t   datLen;
uint32_t   userVal;
//EVENT_PAK
uint16_t   eventRecordNum;
uint16_t   datLen;
//EVENT_RECORD
uint32_t   ledHwId;
uint32_t   subHwId;
uint32_t   swVer;
uint32_t   confVer;
uint8_t    reserved;
int8_t     errCode;
uint16_t   current[3];
```

- 2) 服务器收到后返回设备时间信息。

```
uint8_t    token;
```

```

uint8_t    cmd = CMD_CHECKIN|0x80
uint8_t    ver;
uint8_t    checksum;
uint32_t    swVer;
uint16_t    cmdSeq;
uint16_t    datLen;
uint32_t    userVal = epoch seconds

```

3.3 CMD_ALARM 命令

该命令由节点设备发现故障时汇报给服务器，服务器收到该命令无需回应。节点发送的数据格式同 3.2 CMD_CHECKIN

3.4 CMD_POWER_ON 命令

该命令当节点设备上电或重启时报给服务器，服务器收到该命令后需将系统时间返回给节点设备用于设备本地时间的同步。服务器返回的数据格式同 3.2 CMD_CHECKIN

举例：

1) 设备发送的数据格式为：CMD_FRAME

```

uint8_t    token;
uint8_t    cmd = CMD_POWER_ON
uint8_t    ver;
uint8_t    checksum;
uint32_t    swVer;
uint16_t    cmdSeq;
uint16_t    datLen;
uint32_t    userVal;

```

2) 服务器收到后返回设备时间信息。

```

uint8_t    token;
uint8_t    cmd = CMD_POWER_ON |0x80
uint8_t    ver;
uint8_t    checksum;
uint32_t    swVer;
uint16_t    cmdSeq;
uint16_t    datLen;
uint32_t    userVal = epoch seconds

```

3.5 CMD_CHECK_FW 命令

该命令设备节点发给服务器，检查是否有新的固件用于更新。服务器收到该命令后，如果有新的固件可以通过 FIRMWARE_PAK 格式数据帧返回新固件的信息，供设备自行决定是否升级。如果没有可升

级版本，可不做任何反馈，直接丢弃该命令即可。

举例：

1) 设备发送的数据格式为：CMD_FRAME

```
uint8_t    token;
uint8_t    cmd = CMD_CHECK_FW
uint8_t    ver;
uint8_t    checksum;
uint32_t   swVer;
uint16_t   cmdSeq;
uint16_t   datLen;
uint32_t   userVal;
```

2) 如果有可更新固件，服务器返回的数据格式为：CMD_FRAME

```
uint8_t    token;
uint8_t    cmd = CMD_CHECK_FW|0x80
uint8_t    ver;
uint8_t    checksum;
uint32_t   swVer;
uint16_t   cmdSeq;
uint16_t   datLen;
uint32_t   userVal;
//FIRMWARE_INFO_DAT
uint32_t   swVer;
uint16_t   fwLen;
uint8_t    fwChecksum;
uint8_t    reserved;
```

4. 参数配置

设备的参数配置通过专门的配置程序 trafficLightConf 来实现，程序的运行环境为 linux-x86-64，依赖的运行库为 libpaho-mqtt3a。部分 linux 的发行版可能缺省并未安装该运行库，如果程序运行时提示“./trafficLightConf: error while loading shared libraries: libpaho-mqtt3a.so.1: cannot open shared object file: No such file or directory ”即表示未找到运行库。

此时需要用户自行安装 libpaho-mqtt3a 运行库。在 debian/Ubuntu 环境下可通过命令“sudo apt-get install libpaho-mqtt3a”安装。

参数配置需要三个文件：

trafficLightConf	主运行程序
mqtt.ini	存放服务器登录信息，告诉 trafficLightConf 如何登录服务器
trafficLight.ini	存放信号灯监控设备的配置参数，该文件的内容将被配置到对应的设备

4.1 mqtt.ini 结构解释

mqtt.ini 存放服务器登录信息，告诉 trafficLightConf 如何登录服务器，具体参数解释如下：

```
[mqtt]
mqttServerIp=222.111.123.124      ;MQTT 服务器 IP 地址
```

```

mqttServerPort=8166                ;MQTT 服务器端口
mqttUserName=aabb                  ;MQTT 服务器登录用户名（最长 8 位）
mqttPassword=12345678             ;MQTT 服务器登录密码（最长 8 位）
mqttTopicPrefix=trafficLight      ;topic 前缀（一般不需要修改）
networkCode=0                     ;网络号（一般不需要修改）
stationCode=0                     ;站点号（一般不需要修改）

```

4.2 trafficLight.ini 结构解释

trafficLight.ini 用于存放信号灯监控设备的配置参数，告诉 trafficLightConf 用什么参数去配置信号灯故障监控设备，具体参数解释如下：

```

[nodeConfig]
confVer=26040400                  ;配置版本号（16 进制格式 YYMMDDBB(注*)，用户可自定义）
mqttServerUserIp=223.11.22.33     ;MQTT 服务器 IP 地址
mqttServerPort=2233               ;MQTT 服务器端口号
mqttUserName=aabb                 ;MQTT 服务器登录用户名（最长 8 位）
mqttPassword=h123456              ;MQTT 服务器登录密码（最长 8 位）
networkCode=0                    ;网络号（一般不需要修改）
stationCode=0                    ;站点号（一般不需要修改）
mqttTopicPrefix=trafficLight      ;topic 前缀（一般不需要修改）
beginMin=0                        ;工作起始分钟（暂未启用）
beginHour=0                      ;工作起始小时（暂未启用）
endMin=59                        ;工作结束分钟（暂未启用）
endHour=23                       ;工作结束小时（暂未启用）
enLed=7                          ;使能的灯组（暂未启用）
checkinMin=2                     ;签到周期（分钟）
gapSec=2                         ;信号灯切换间隙（请保持缺省值）
ledMaxPeriodSecR=100             ;红灯最长周期秒数，若红灯亮灯时间超过此值，上报红灯超时错误
ledMaxPeriodSecY=5               ;黄灯最长周期秒数，若黄灯亮灯时间超过此值，上报黄灯超时错误
ledMaxPeriodSecG=120             ;绿灯最长周期秒数，若绿灯亮灯时间超过此值，上报绿灯超时错误
powerLossSec=200                 ;断电最长等待秒数，若三个信号灯同时灭灯时间超过此值，上报断电
ledDimThresholdR=200             ;红灯缺亮阈值（暂未启用）
ledDimThresholdY=200             ;黄灯缺亮阈值（暂未启用）
ledDimThresholdG=200             ;绿灯缺亮阈值（暂未启用）

```

注*：YYMMDDBB，是 16 进制编码方式，YY=年，MM=月，DD=天，BB=当天编号。例如：26040102 表示 26 年 04 月 01 日的 02 号配置。

4.3 trafficLightConf 使用方法

trafficLightConf 首先读取 mqtt.ini 获取登录服务器的参数，然后读取 trafficLight.ini 内容获得要发出的配置信息。trafficLight.ini 中的内容作为配置缺省模板，用户可通过附加选项参数的方式修改其中的配置内容（注：选项参数会覆盖对应的参数项，但不会保存到 ini 文件）。trafficLightConf 的使用方法如下：

trafficLightConf <hwId> [options]

hwID 是要配置设备的硬件编码,

options 是选项参数, 可用的选项参数如下:

--verbose	; 运行时打印显示配置参数及过程
--help	; 显示帮助信息
--version	; 显示版本信息
--confVer	; 强制修改 trafficLight.ini 中的 confVer 字段
--mqttServerUserIp	; 强制修改 trafficLight.ini 中的 mqttServerUserIp 字段
--mqttServerPort	; 强制修改 trafficLight.ini 中的 mqttServerPort 字段
--mqttUserName	; 强制修改 trafficLight.ini 中的 mqttUserName 字段
--mqttPassword	; 强制修改 trafficLight.ini 中的 mqttPassword 字段
--networkCode	; 强制修改 trafficLight.ini 中的 networkCode 字段
--stationCode	; 强制修改 trafficLight.ini 中的 stationCode 字段
--mqttTopicPrefix	; 强制修改 trafficLight.ini 中的 mqttTopicPrefix 字段
--beginMin	; 强制修改 trafficLight.ini 中的 beginMin 字段
--beginHour	; 强制修改 trafficLight.ini 中的 beginHour 字段
--endMin	; 强制修改 trafficLight.ini 中的 endMin 字段
--endHour	; 强制修改 trafficLight.ini 中的 endHour 字段
--enLed	; 强制修改 trafficLight.ini 中的 enLed 字段
--checkinMin	; 强制修改 trafficLight.ini 中的 checkinMin 字段
--gapSec	; 强制修改 trafficLight.ini 中的 gapSec 字段
--ledMaxPeriodSecR	; 强制修改 trafficLight.ini 中的 ledMaxPeriodSecR 字段
--ledMaxPeriodSecY	; 强制修改 trafficLight.ini 中的 ledMaxPeriodSecY 字段
--ledMaxPeriodSecG	; 强制修改 trafficLight.ini 中的 ledMaxPeriodSecG 字段
--powerLossSec	; 强制修改 trafficLight.ini 中的 powerLossSec 字段
--ledDimThresholdR	; 强制修改 trafficLight.ini 中的 ledDimThresholdR 字段
--ledDimThresholdY	; 强制修改 trafficLight.ini 中的 ledDimThresholdY 字段
--ledDimThresholdG	; 强制修改 trafficLight.ini 中的 ledDimThresholdG 字段

程序输出信息如下

```
./trafficLightConf 1114006C
waiting for connection
Successful connection
Subscribing to topic trafficLight/0/0/+/U
for client trafficLightConf using QoS1

Successful connection
pub topic =trafficLight/0/0/1114006C/D, cmd=0x20 len=148
Waiting for publication...
Subscribe succeeded
Message with token value 2 delivery confirmed
Successful disconnection
finished.
```

配置过程仅仅是通过 MQTT 服务器向对应的信号灯监控设备发出配置信息，成功发出并不代表设备一定会被正确配置。设备正确收到新的配置后，会主动重新启动，此时服务器应该会收到设备的上电信息，可通过设备上报的 confVer 字段与发出 confVer 字段是否相同，来判断配置成功与否。

命令举例：

(1) `./traffiLightConf 11140000`

使用 trafficLight.ini 中的缺省模板参数，去配置 hwId=11140000 的设备。这是最简单的应用，可将配置信息修改好后，通过脚本调用该命令实现批量配置更新。

(2) `./traffiLightConf 11140000 --confVer 26050100 --powerLossSec 300`

临时修改 trafficLight.ini 中的 confVer 和 powerLossSec 参数，其余使用模板缺省参数去修改 11140000 设备的配置。该模式适合临时调试参数（注：选项参数会覆盖对应的参数项，但不会保存到 ini 文件）。

5. 附录

1.1 校验算法

该算法用于命令帧的校验和固件校验和的生成

```
/**
 * @brief calculate the checksum of the buf
 * @author wuj (2023/12/20 13:04:39)
 *
 * @param buf    - input the value to calculate checksum
 * @param len    - input the length of the buf
 *
 * @return uint8_t - return the checksum
 */
uint8_t calculateChecksum(const uint8_t *buf, uint16_t len)
{
    uint16_t i;
    uint8_t checksum;

    checksum = 0;
    for (i = 0; i < len; i++) {
        checksum += buf[i];
    }

    return checksum;
}
```